

Scribed by:

1. Viha Singla (2022MT61972)
2. Nagure Kalyani Paramanand (2022MT61983)
3. Khushi Gupta (2022MT61973)
4. Raman Jakhar (2022MT11941)

1 Overview

In the previous lecture, we studied key aspects of Turing Machines (TMs) and their computational power. We introduced **Nondeterministic Turing Machines (NTMs)**, whose transition function allows multiple moves for a single configuration. An NTM accepts an input if at least one computation path reaches an accept state. Despite this flexibility, NTMs are **equivalent in power to Deterministic Turing Machines (DTMs)**:

- **Theorem 1:** Every NTM has an equivalent DTM.
- **Theorem 2:** A language is recognizable by a DTM if and only if it is recognizable by an NTM.

This equivalence relies on simulating an NTM using a DTM that systematically explores all computation paths (via a three-tape DTM implementing breadth-first search).

We also saw that TMs can decide **graph connectivity** by encoding the graph on the tape and performing a BFS-like traversal to mark reachable vertices. If all vertices are marked, the graph is connected; otherwise, it is not.

Finally, we discussed the **Church–Turing Thesis**, which asserts that any effectively computable function can be computed by a Turing Machine. This thesis, supported by the equivalence of models like lambda calculus and recursive functions, defines the theoretical limits of computation.

2 Decidability and Decision Problems

In this lecture, we continue the study of Turing Machines by exploring the relationship between **Turing-recognizable** and **Turing-decidable** languages and analyzing several fundamental decision problems.

2.1 Turing-Recognizable and Turing-Decidable Languages

A language L is said to be **Turing-Recognizable (TR)** if there exists a Turing Machine M that accepts every string $w \in L$ and either rejects or loops indefinitely for $w \notin L$. Formally, L is Turing-recognizable if $L = L(M)$ for some TM M .

A language L is **Turing-Decidable (TD)** if there exists a Turing Machine (called a *decider*) that halts on all inputs and correctly accepts or rejects depending on whether the input string belongs to L .

A **Turing-decidable** language is always **Turing-recognizable**, since a decider necessarily halts and accepts all strings in the language. However, the converse is not true: there exist languages that are recognizable but not decidable, as their corresponding Turing Machines may loop forever on certain inputs.

Note: If a language is not Turing-recognizable, it is particularly noteworthy, since it lies outside the scope of algorithmic computation. Such languages highlight the limitations of what Turing Machines can compute.

2.2 Decision Problems on Formal Languages

We now study several classical decision problems and determine whether the corresponding languages are Turing-decidable.

2.2.1 Acceptance Problem for DFA

Problem. Given a deterministic finite automaton (DFA) B and a string w , determine whether B accepts w .

Formally, we define the language:

$$A_{DFA} = \{\langle B, w \rangle \mid B \text{ is a DFA and } B \text{ accepts } w\}.$$

Each element of A_{DFA} is an **encoding** of a pair (B, w) , where B is a finite automaton and w is an input string over its alphabet. Thus, A_{DFA} represents the set of all machine-input pairs for which the DFA accepts the input.

Intuition. This problem asks whether we can build a Turing Machine that takes as input a *description* of some DFA B and a string w , and decides (in a finite number of steps) if w would be accepted by B . In other words, the Turing Machine must *simulate* the DFA's behavior on the input w .

How does the simulation work? The input to our Turing Machine M is an encoding $\langle B, w \rangle$, which includes:

- The states, alphabet, start state, accepting states, and transition function of B , and
- The string w that is to be tested.

The TM M then behaves as follows:

1. Initialize a variable to store the current state of B (starting with B 's start state).
2. Scan the string w symbol by symbol.
3. For each symbol, look up B 's transition rule from the current state and update the current state accordingly.
4. After the last symbol of w is read, check whether the current state is an accepting state of B .
 - If it is, M **accepts**.
 - Otherwise, M **rejects**.

Since the DFA has a finite number of states and the input string w is finite, this process always halts. Hence, the Turing Machine M **decides** the language A_{DFA} .

Theorem. A_{DFA} is Turing-decidable.

2.2.2 Acceptance Problem for NFA

Problem. Given a nondeterministic finite automaton (NFA) B and a string w , decide whether B accepts w .

Formally, we define:

$$A_{NFA} = \{\langle B, w \rangle \mid B \text{ is an NFA and } B \text{ accepts } w\}.$$

Explanation. An NFA accepts an input string if there exists *at least one* computation path that leads to an accepting state. However, simulating all possible nondeterministic paths directly on a Turing Machine would be cumbersome. Instead, we can make use of the fact that every NFA has an equivalent DFA that recognizes the same language.

Approach.

1. Convert the given NFA B into an equivalent DFA D using the subset construction method.
2. Use the Turing Machine that decides A_{DFA} to check whether D accepts w .

Since both the conversion process and DFA simulation are finite and algorithmic, the entire procedure halts.

Theorem. A_{NFA} is Turing-decidable.

2.2.3 Emptiness Problem for DFA

Problem. Given a DFA B , decide whether the language recognized by B is empty, i.e., whether $L(B) = \emptyset$.

Formally,

$$E_{DFA} = \{\langle B \rangle \mid L(B) = \emptyset\}.$$

Explanation. A DFA's language is empty if no accepting state is reachable from its start state. This can be verified by analyzing B 's transition structure rather than testing infinitely many strings.

Approach.

1. View the DFA as a directed graph, where states are vertices and transitions are edges.
2. Starting from the start state, perform a traversal (e.g., BFS).
3. If any accepting state is reachable, then $L(B) \neq \emptyset$; otherwise, $L(B) = \emptyset$.

This traversal is guaranteed to halt because the DFA has finitely many states.

Theorem. E_{DFA} is Turing-decidable.

2.2.4 Equivalence Problem for DFA

Problem. Given two DFAs B_1 and B_2 , decide whether they recognize the same language:

$$L(B_1) = L(B_2).$$

Formally,

$$EQ_{DFA} = \{\langle B_1, B_2 \rangle \mid L(B_1) = L(B_2)\}.$$

Explanation. Two DFAs are equivalent if they accept exactly the same set of strings. To check this, we can construct a new DFA that accepts strings where their behaviors differ, and then test whether that DFA's language is empty.

Approach.

1. Construct a new DFA B^* such that:

$$L(B^*) = (L(B_1) \cap \overline{L(B_2)}) \cup (L(B_2) \cap \overline{L(B_1)}).$$

2. Intuitively, B^* accepts exactly those strings that are accepted by one DFA but rejected by the other.
3. Apply the **emptiness test** (from E_{DFA}) on B^* :
 - If $L(B^*) = \emptyset$, then $L(B_1) = L(B_2)$ and the DFAs are equivalent.
 - Otherwise, there exists at least one string on which B_1 and B_2 differ.

Why can B^* be constructed easily? DFAs are known to be closed under complement, union, and intersection, etc. This means that if we have DFAs for $L(B_1)$ and $L(B_2)$, then DFAs for $\overline{L(B_1)}$, $\overline{L(B_2)}$, $(L(B_1) \cap \overline{L(B_2)})$, and $(L(B_2) \cap \overline{L(B_1)})$ can all be systematically constructed. Therefore, the combined DFA B^* for the symmetric difference can be built mechanically.

Theorem. EQ_{DFA} is Turing-decidable.

2.2.5 Acceptance Problem for Regular Grammars (Regular Grammar / Regular Expression)

Problem. Given a regular grammar (or equivalently a regular expression or right-/left-linear grammar) R and a string w , decide whether R generates w .

Formally,

$$A_{REG} = \{\langle R, w \rangle \mid R \text{ is a regular grammar and } w \in L(R)\}.$$

Intuition. Regular grammars are equivalent in expressive power to finite automata. Thus to decide whether w belongs to $L(R)$ we can convert the grammar to an equivalent DFA (or NFA) and simulate it on w .

Approach.

1. Convert the given regular grammar R to an equivalent NFA N .
2. (Optional) Convert N to an equivalent DFA D using the subset construction.
3. Simulate the resulting finite automaton on the input string w .
 - If the automaton accepts, **accept**.
 - Otherwise **reject**.

Because the conversion steps are finite and the automaton simulation halts (finite machine on finite input), this procedure always halts.

Theorem. A_{REG} is Turing-decidable.

2.2.6 Acceptance Problem for CFG

Problem. Given a context-free grammar G and a string w , decide whether G generates w .

$$A_{CFG} = \{\langle G, w \rangle \mid w \in L(G)\}.$$

Intuition. To determine whether w is in $L(G)$, we can systematically generate all strings that G can produce up to a certain length and then check whether w appears among them. Since any derivation producing w cannot be longer than $2|w| - 1$ steps in a Chomsky Normal Form (CNF) grammar, this process always halts.

Approach.

1. Convert the given CFG G into an equivalent grammar G' in **Chomsky Normal Form (CNF)**.
2. Enumerate all possible derivations in G' up to length $\leq |w|$.
3. For each derivation, generate the corresponding string.
4. If any generated string equals w , then:

- **Accept.**

Otherwise:

- **Reject.**

Why this halts: There are finitely many derivations of length $\leq |w|$ because the grammar and alphabet are finite. Thus, the enumeration eventually terminates after generating all possible strings of that length.

Theorem. A_{CFG} is Turing-decidable.

2.2.7 Emptiness Problem for CFG

Problem. Given a CFG G , decide whether $L(G) = \emptyset$.

$$E_{CFG} = \{\langle G \rangle \mid L(G) = \emptyset\}.$$

Approach (marking algorithm). Use the standard algorithm that marks all variables that can derive some terminal string:

1. Initialize the set M of *generating* variables to be empty.
2. Repeat until no change:
 - For each production $A \rightarrow \alpha$ of G , if every symbol of α is either a terminal or already in M , add A to M .
3. After saturation, check whether the start variable S is in M .
 - If $S \in M$, then $L(G) \neq \emptyset$ (there exists some derivation to a terminal string).
 - If $S \notin M$, then $L(G) = \emptyset$.

Correctness sketch: Variables in M are precisely those that can derive some terminal string (by induction on the number of marking steps). If S cannot be marked, there is no derivation from S to a terminal string.

Theorem. E_{CFG} is Turing-decidable.

2.2.8 Equivalence Problem for CFG

Problem. Given two CFGs G_1 and G_2 , decide whether $L(G_1) = L(G_2)$.

$$EQ_{CFG} = \{\langle G_1, G_2 \rangle \mid L(G_1) = L(G_2)\}.$$

Observation / Fact. The equivalence problem for arbitrary context-free grammars is **not** Turing-decidable.

Reason (informal sketch).

- CFLs are not closed under complement, so the standard DFA method (build symmetric difference and test emptiness) does not carry over.
- More strongly, one can reduce known undecidable problems (for example certain PCP/ATM style encodings) to CFG equivalence; in particular, it is known from formal language theory that testing equivalence of two arbitrary CFGs is undecidable.

Theorem. EQ_{CFG} is not Turing-decidable.

Remark. There are decidable special cases: for instance, equivalence of deterministic context-free languages is decidable under specific models, but the general CFG equivalence problem remains undecidable.

References

- [1] P. Linz, *An Introduction to Formal Languages and Automata, 6th Edition*, Jones & Bartlett Learning, 2016.